

```
class Banner
```

```
  attr_accessible :horiz, :link, :visible, :image, :position  
  has_attached_file :image, styles: { vert: '220'
```

```
  before_create :assign_position
```

```
class Banner < ActiveRecord::Base
```

```
  attr_accessible :horiz, :link, :visible, :image  
  has_attached_file :image, styles: { vert: '220'
```

```
  before_create :assign_position
```

```
  protected
```

```
  def assign_position
```

```
    max = Banner.maximum(:position)
```

```
    self.position = max ? max + 1 : 0
```

FULL STACK DEVELOPMENT

REACT - ADVANCED

AULA 03

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	6
O QUE VOCÊ VIU NESTA AULA?	9
REFERÊNCIAS.....	10

EMANSP

O QUE VEM POR AÍ?

Prepare-se para explorar um novo nível de gerenciamento de estado em suas aplicações React. Nesta aula, você aprenderá a utilizar a Context API para simplificar e otimizar a gestão de estado, especificamente focando em um projeto de e-commerce. Imagine ter um carrinho de compras que responde instantaneamente às ações do usuário, sem a complexidade de passar dados manualmente entre componentes. Esse é o poder da Context API, e é exatamente isso que você dominará.

Vamos começar com uma visão geral do projeto de e-commerce, onde o principal objetivo é implementar a Context API para gerenciar o carrinho de compras. Primeiro, faremos um tour pelo projeto existente, destacando os componentes visuais já implementados, especialmente a página de carrinho e o carrinho suspenso. Você verá como identificar quais componentes precisarão estar conectados por meio da Context API.

Com essa base, configuraremos o CartContext usando o React Context. Definiremos o estado inicial e a estrutura básica necessária para compartilhar dados do carrinho entre os componentes. Em seguida, criaremos o CartProvider, que envolverá os componentes que precisam acessar os dados do carrinho, discutindo como ele funciona para passar dados e funções aos componentes filhos.

A seguir, focaremos na manipulação do estado do carrinho com a Context API. Implementaremos funções para adicionar e remover itens do carrinho, além de ajustar quantidades. Estas funções serão integradas com a interface, conectando-se aos botões e ações nos componentes de produto e carrinho. Você verá, na prática, como o contexto permite atualizar o estado do carrinho de maneira centralizada e eficiente, evitando o uso de props drilling.

Finalmente, refinaremos nosso código e discutiremos melhores práticas para usar a Context API. Recapitularemos o que foi aprendido, garantindo que você entenda profundamente como a Context API simplifica a gestão de estado e melhora a manutenção e escalabilidade de suas aplicações React.

HANDS ON

Prepare-se para uma imersão prática no gerenciamento de estado em React com a Context API! Nos próximos três vídeos, exploraremos passo a passo como implementar e utilizar essa ferramenta para gerenciar o carrinho de compras em um projeto de e-commerce. Você verá como a Context API pode simplificar significativamente a gestão de estado, tornando seu código mais limpo e eficiente. Vamos começar essa jornada juntos e dominar a Context API em três etapas envolventes e instrutivas.

No primeiro vídeo, daremos o pontapé inicial com uma introdução à Context API, explicando sua importância e como ela pode revolucionar a gestão de estado nas suas aplicações React. Você conhecerá o projeto de e-commerce e entenderá o objetivo de implementar a Context API para gerenciar o carrinho de compras. Faremos um tour pelo projeto existente, destacando os componentes visuais, especialmente a página de carrinho e o carrinho suspenso. Identificaremos quais componentes precisarão estar conectados via Context API. Em seguida, configuraremos o CartContext, definindo o estado inicial e a estrutura básica para compartilhar dados do carrinho. Por fim, criaremos o CartProvider, discutindo como ele passa dados e funções aos componentes filhos.

No segundo vídeo, entraremos na implementação prática da lógica para adicionar e remover itens do carrinho usando a Context API. Implementaremos funções no contexto para adicionar itens ao carrinho, remover itens e ajustar quantidades. Essas funções serão integradas com a interface, conectando-se aos botões e ações nos componentes de produto e carrinho. Você verá, na prática, como o contexto permite atualizar o estado do carrinho de maneira centralizada e eficiente. Além disso, faremos testes para garantir que as funcionalidades estão operando corretamente, e discutiremos como a Context API facilita a gestão de estado sem a necessidade de props drilling.

No terceiro e último vídeo, focaremos em refinar nosso código e discutir melhores práticas para o uso da Context API. Recapitularemos tudo o que foi aprendido, desde a configuração inicial até a manipulação avançada do estado do carrinho. Discutiremos estratégias para otimizar ainda mais a performance e a

organização do código, garantindo que sua aplicação esteja pronta para escalar. Além disso, você aprenderá dicas valiosas para manter o código limpo e eficiente, assegurando que a gestão de estado permaneça simplificada e robusta.

EMENDAS

SAIBA MAIS

Prop Drilling é mesmo um problema?

Prop drilling é uma técnica no desenvolvimento de aplicações React onde as props são passadas manualmente de um componente pai para vários níveis de componentes filhos. Esse método é utilizado para compartilhar dados ou funções entre componentes que não são diretamente relacionados, mas que precisam se comunicar. Isso pode levar a um código mais difícil de manter e entender, já que muitos componentes intermediários são forçados a aceitar e repassar props desnecessárias.

Isso pode acontecer quando a estrutura da aplicação não foi projetada para uma gestão eficiente de estado ou quando a comunicação entre componentes é necessária, mas a aplicação não utiliza ferramentas mais adequadas, como a Context API ou bibliotecas de gerenciamento de estado como Redux.

Apesar dos desafios, existem situações onde o prop drilling pode fazer sentido e ser aceitável, especialmente em aplicações pequenas ou em casos onde a passagem de dados é limitada a um ou dois níveis. Em projetos menores, a complexidade adicional de implementar uma solução de gerenciamento de estado mais robusta pode não ser justificada. Além disso, quando a estrutura dos componentes é clara e bem definida, e os dados são passados apenas por um ou dois níveis, o prop drilling pode ser uma solução rápida e eficaz.

Mas se você optar por utilizar o prop drilling, alguns cuidados são essenciais para manter o código limpo e manutenível:

- Documentação clara: documente bem a finalidade das props passadas e os componentes que as utilizam. Isso ajudará outros(as) desenvolvedores(as) (ou você mesmo(a) no futuro) a entenderem o fluxo de dados.
- Evite níveis profundos: tente limitar o número de níveis pelos quais as props são passadas. Se perceber que está passando por mais de dois níveis, considere outras abordagens, como a Context API.
- Reutilização de componentes: mantenha os componentes reutilizáveis e independentes sempre que possível, evitando que se tornem muito dependentes das props passadas.

- Considere alternativas: em aplicações maiores ou mais complexas, considere o uso de Context API, Redux, ou outras bibliotecas de gerenciamento de estado que possam centralizar e simplificar a passagem de dados.

Ferramentas para gestão de estados em React

No desenvolvimento de aplicações React, gerenciar o estado de maneira eficiente é crucial para garantir a integridade e a manutenção do código. Diversas ferramentas foram criadas para auxiliar nessa tarefa, cada uma com suas particularidades. Vamos explorar três das principais opções: Redux, Zustand e Recoil, entendendo suas arquiteturas e como elas podem ser aplicadas em projetos React.

Redux

Redux é uma das bibliotecas mais antigas e amplamente utilizadas para gerenciamento de estado em aplicações JavaScript, especialmente em React. Sua arquitetura se baseia em três princípios fundamentais: um único store, estado imutável e mudanças previsíveis. O estado da aplicação é armazenado em um objeto global, o store, que é imutável. As mudanças no estado são feitas por meio de ações que descrevem o que deve mudar, e reducers que especificam como as mudanças devem ocorrer.

A principal vantagem do Redux é a previsibilidade do estado, facilitada por suas ferramentas de desenvolvimento, que permitem rastrear cada alteração de estado e ação disparada. Isso torna a depuração mais fácil e o comportamento da aplicação mais transparente. Contudo, a configuração inicial pode ser verbosa, com muito boilerplate, e sua curva de aprendizado é mais acentuada, especialmente para iniciantes.

Zustand

Zustand é uma biblioteca recente que oferece uma abordagem mais simples e leve para o gerenciamento de estado. Diferente do Redux, que usa um store global, Zustand permite a criação de stores locais e independentes, o que pode ser mais adequado para projetos menores ou componentes isolados. A configuração do Zustand é minimalista, sem a necessidade de muita configuração inicial ou código boilerplate.

A arquitetura do Zustand é baseada em hooks do React, tornando a integração direta e natural para desenvolvedores React. A simplicidade e a flexibilidade são seus principais atrativos, permitindo que os(as) desenvolvedores(as) definam e usem o estado de maneira muito mais direta e menos cerimoniosa. No entanto, por ser uma ferramenta nova, ainda possui menos recursos e uma comunidade menor comparada ao Redux, o que pode resultar em menos suporte e plugins disponíveis.

Recoil

Recoil é uma biblioteca desenvolvida pelo Facebook para gerenciar estados de maneira mais intuitiva em aplicações React. Sua arquitetura é baseada em átomos e seletores. Átomos são as menores unidades de estado que podem ser compartilhadas entre componentes, enquanto seletores são funções derivadas do estado que podem ser usadas para calcular novos valores baseados nos átomos.

Ela se integra perfeitamente com os hooks do React, oferecendo uma maneira natural e eficiente de gerenciar estados complexos. A principal vantagem do Recoil é sua eficiência em grandes aplicações, graças ao seu modelo de dependência de estado que permite atualizações granulares. Como é uma ferramenta mais recente, Recoil ainda está em crescimento, com uma base de usuários e documentação que estão em constante expansão.

Cada uma dessas ferramentas oferece uma abordagem distinta para o gerenciamento de estado em React, com suas próprias vantagens e desvantagens. Redux é robusto e bem documentado, ideal para projetos grandes e complexos. Zustand é leve e flexível, ótimo para projetos menores ou partes específicas da aplicação. Recoil é intuitivo e eficiente, especialmente em grandes aplicações com estados complexos.

A escolha da ferramenta depende das necessidades específicas do seu projeto e da familiaridade da equipe com a ferramenta. Explorar e experimentar essas opções permitirá que você encontre a solução mais adequada para o seu contexto de desenvolvimento.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos como simplificar o gerenciamento de estado em aplicações React utilizando a Context API, com um foco especial em um projeto de e-commerce. Ao longo dos três vídeos, passamos por várias etapas práticas e conceitos essenciais que agora lhe permitem desenvolver aplicações mais eficientes e organizadas.

Começamos com uma introdução ao projeto de e-commerce, onde nosso principal objetivo era implementar a Context API para gerenciar o carrinho de compras. Fizemos uma revisão do projeto existente, identificando os componentes que precisavam se comunicar diretamente com o estado do carrinho. Com isso em mente, configuramos o CartContext, definindo o estado inicial e criando o CartProvider, que envolve os componentes que precisam acessar os dados do carrinho.

A seguir, focamos na implementação das funções essenciais para gerenciar o estado do carrinho. Desenvolvemos funções para adicionar e remover itens, bem como ajustar quantidades, integrando essas funções diretamente com a interface do usuário. Testamos essas funcionalidades para garantir que o estado do carrinho fosse atualizado corretamente, demonstrando como a Context API centraliza e simplifica a gestão de estado.

Finalizamos a aula discutindo refinamentos e melhores práticas para usar a Context API. Recapitulamos os conceitos e técnicas aprendidas, destacando a importância de manter o código limpo e eficiente. Abordamos estratégias de otimização e mostramos como a Context API facilita a manutenção e escalabilidade de projetos React.

REFERÊNCIAS

ALURA. **Estados globais: diferenças entre Redux e Context API**. Disponível em: <https://www.alura.com.br/artigos/estados-globais-diferencas-redux-context-api>. Acesso em: 25 jun. 2024.

REACT. **createContext**. Disponível em: <https://react.dev/reference/react/createContext>. Acesso em: 25 jun. 2024.

REACT. **Reusing Logic with Custom Hooks**. Disponível em: <https://react.dev/learn/reusing-logic-with-custom-hooks#when-to-use-custom-hooks>. Acesso em: 25 jun. 2024.

PALAVRAS-CHAVE

State Management, React Context, Context Provider, useContext Hook, Global State, Prop Drilling.

EMEND

POSTECH